

임장혁

문제를 기능이 아닌 시스템 관점에서 해결하는 백엔드 개발자입니다.

단순 기능 구현보다 “왜 이런 문제가 발생하는가?” “왜 이런 구조가 필요한가?” 를 고민하며, 실험과 수치 기반으로 시스템을 개선하는 개발을 지향합니다.

깃허브: github.com/imjanghyeok 블로그: dlaljh.tistory.com 이메일: qkdrhkg23@naver.com

관심사: Java · Spring Boot · TypeScript · NestJS · LLM

업무 스타일

기능을 바로 구현하기보다 문제가 발생하는 구조와 상태의 소유자를 먼저 정의합니다.

- 문제 재정의 → 대안 비교 → 구현 → 측정 → 한계 기록
- 상태의 수명과 저장소를 분리하고, 비동기 흐름의 제어 지점을 명확히 둡니다.
- 팀 작업에서는 이슈 문서화, 리뷰 규칙, 품질 게이트로 의사결정 맥락을 공유합니다.

강점

- **State Modeling**
Redis unread, TTL session, Scheduler 상태 전이
- **Flow Control**
Kafka ordering, WebSocket fan-out, LLM 종료 정책
- **Responsibility**
도메인 객체, Consumer, 서버 정책의 변경 이유 분리
- **Validation**
실험, 성능 수치, CI/E2E 기반 회귀 검증

기술 스택

Backend Java · Spring Boot · NestJS · Node.js · TypeScript

Message / Cache Kafka · Redis · WebSocket · STOMP

Data MySQL · PostgreSQL · MongoDB · JPA · TypeORM

Infra Docker · Docker Compose · GitHub Actions · AWS · NCP · Nginx

PROJECT 01 · SYNAPSE

01. Synapse

AI 면접 시뮬레이션 서비스

이 프로젝트를 한 줄로 설명하면

LLM 비결정성을 서버 정책 기반으로 제어한 AI 면접 시뮬레이션 프로젝트

1. 프로젝트 개요

사용자의 포트폴리오를 기반으로 AI 면접관이 개인화된 질문과 피드백을 제공하는 서비스입니다.

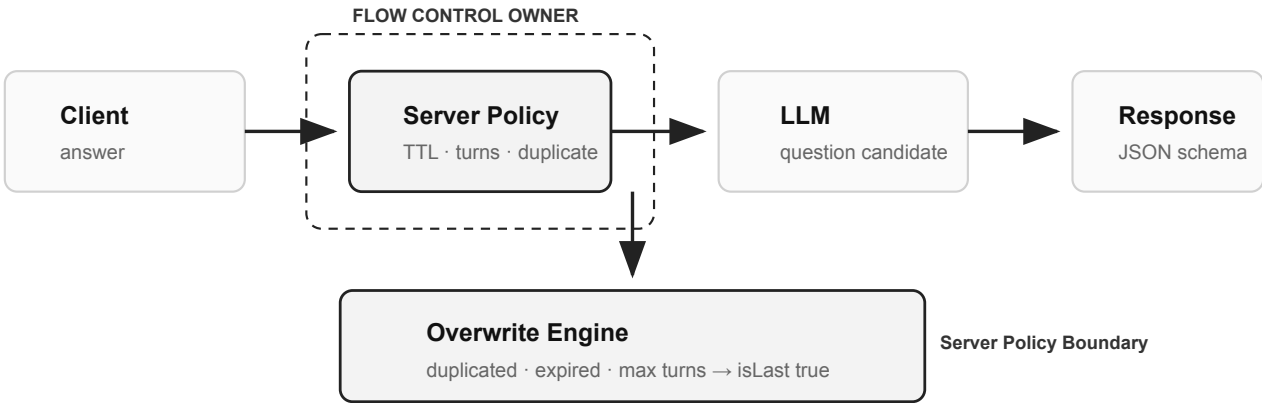
내 역할: 면접 진행 API, 질문 생성 흐름, 세션 상태 관리, 문서/면접 도메인 삭제 정책, TypeORM Migration, CI/E2E, Docker 배포 최적화.

핵심 문제

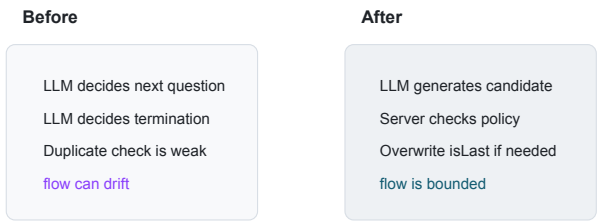
LLM은 질문을 생성할 수 있지만, 면접의 종료 조건과 중복 방어까지 안정적으로 보장하지는 못했습니다. 그래서 LLM 출력은 후보로 두고, 서비스 정책은 서버가 소유하도록 설계했습니다.

15.1% 질문 중복률 첫 질문 100개 기준	5배 삭제 성능 개선 ORM remove 의존 제거 + DB Cascade	81% 메모리 절감 백엔드 실행 메모리 247MB → 46MB	4.14 / 5 사용자 피드백 최종 발표 14명 응답 기준
--	---	--	---

Measurement note: 질문 다양성은 동일 입력 기준 100회 호출 결과, 사용자 피드백은 최종 발표 피드백 14명 응답 기준입니다. 성능 수치는 팀 테스트 환경 기준으로 해석합니다.



LLM은 질문 후보를 생성하고, 서버 정책이 중복·종료·재시도 흐름을 최종 결정합니다.



면접 중 임시 상태는 TTL 세션으로 분리하고, 만료 처리는 MinHeap 기반 Scheduler가 담당합니다.

IMPLEMENTATION EVIDENCE · SIMPLIFIED FROM IMPLEMENTATION

MinHeapScheduler

```
class MinHeapScheduler {
  schedule(key, expiresAt)
  tick(): while heap.peek().expiresAt <=
now
    store.delete(expired.key)
}
```

세션마다 timer를 만들지 않고, 가장 먼저 만료될 key만 확인해 정리 비용을 제한했습니다.

Overwrite Engine

```
if (duplicated || ttlExpired ||
maxTurnsReached) {
  response.isLast = true
  mode = "TERMINATION"
}
```

LLM 응답을 그대로 신뢰하지 않고, 서비스 종료 정책을 서버가 덮어쓰도록 분리했습니다.

문제 1. LLM 비결정성으로 면접 흐름이 깨지는 문제

LLM 출력은 질문 후보로만 사용하고, 중복/종료 판단은 서버 정책으로 덮어써 면접 흐름을 제한했습니다.

BEFORE	AFTER
- LLM의 `isLast`와 프롬프트 지시에 의존 - 중복 질문과 무한 진행 가능성 존재	- 서버가 TTL, 최대 턴, 질문 이력으로 최종 판단 - 필요 시 재시도 또는 Termination Mode로 강제 종료

PROBLEM	DECISION
프롬프트만으로는 중복 질문과 자연스러운 종료를 안정적으로 보장하기 어려웠습니다.	LLM은 생성자, 서버는 흐름 제어자로 분리했습니다.

CORE IMPLEMENTATION

Overwrite Engine

Termination Mode

JSON Schema

결론: 면접 흐름이 LLM 출력이 아니라 서버 정책 안에서 동작하도록 바뀌었습니다.

문제 2. 면접 중 임시 상태를 어디에 저장할 것인가

면접 진행 상태를 영속 데이터가 아니라 TTL이 있는 세션 상태로 정의하고, 만료 책임을 Scheduler로 분리했습니다.

PROBLEM

질문 이력, 방문 주제, 톤 수는 면접 중에만 필요한 임시 상태였습니다.

DECISION

상태 보관은 `KeySetStore`, 만료 관리 는 `MinHeapScheduler`가 담당하도록 나눴습니다.

CORE IMPLEMENTATION

KeySetStore

MinHeapScheduler

TTL cleanup

결론: 영속 데이터와 흐름 제어용 임시 상태의 저장 계층을 분리했습니다.

문제 3. 연관 엔티티 삭제 시 정합성과 성능을 동시에 보장하는 방법

연관 삭제 정합성은 DB 제약으로 보장하고, 서비스 계층은 삭제 요청 검증과 응답 책임에 집중시켰습니다.

PROBLEM

문서 삭제 시 자식 데이터 누락과 대량 삭제 성능 저하가 함께 발생할 수 있었습니다.

DECISION

삭제 정합성은 DB에 위임하고, 애플리케이션은 정책 검증에 집중했습니다.

CORE IMPLEMENTATION

onDelete: CASCADE

delete()

Document domain relation

결론: 연관 데이터 삭제 성능을 약 5배 개선하고 고아 데이터 위험을 줄였습니다.

운영 보강

Migration: 운영 환경에서 `synchronize` 의존을 제거하고, 스키마 변경을 TypeORM Migration 파일과 배포 절차 안에서 검토하도록 전환했습니다.

CI/E2E: AI API가 포함된 인터뷰 흐름을 수동 확인에만 맡기지 않기 위해 Lint, Build, Unit, E2E 단계를 분리한 품질 게이트를 구성했습니다.

Docker: 1GB RAM 서버에서 배포 실패 가능성을 줄이기 위해 멀티 스테이지 빌드와 `.dockerignore`를 적용했고, 이미지 크기를 1.2GB에서 250MB로 줄였습니다.

KNOWN LIMITATION

TTL 상태 저장소는 인메모리 기반이므로 서버 재시작과 horizontal scale에 취약합니다. 실제 운영 규모에서는 Redis나 외부 session store로 이전하고, 질문 생성 품질은 프롬프트만이 아니라 검색, 중복 검증 가드레일, 평가 모델을 함께 두는 방향이 필요합니다.

실제 구현

인메모리 TTL 세션 저장소, MinHeapScheduler, Overwrite Engine, TypeORM Cascade 삭제, CI/E2E, Docker 경량화

이후 고려한 방향

Redis 기반 session store, LLM output evaluator, 질문 중복 guardrail, 검색 기반 컨텍스트 주입

회고

LLM 기반 서비스에서 중요한 것은 프롬프트만 잘 쓰는 것이 아니었습니다. 확률적 출력을 어느 범위까지 허용하고, 어느 지점부터 서버 정책이 개입해야 하는지 정하는 일이 서비스 안정성의 핵심이었습니다.

PROJECT 02 · SMILETOGETHER

02. SmileTogether

MSA 기반 협업 메신저

2025.01 - 2025.03 · Backend Lead · Spring Boot · Kafka · MongoDB · Redis · FCM · Docker Compose

이 프로젝트를 한 줄로 설명하면

다중 WebSocket 서버 환경에서 메시지 일관성과 조회 정합성을 보강한 프로젝트

1. 프로젝트 개요

Slack형 협업 메신저를 MSA 구조로 구현한 프로젝트입니다. 채팅, 히스토리, 멤버, 알림 서버의 백엔드 구현과 서버 간 연동을 담당했습니다.

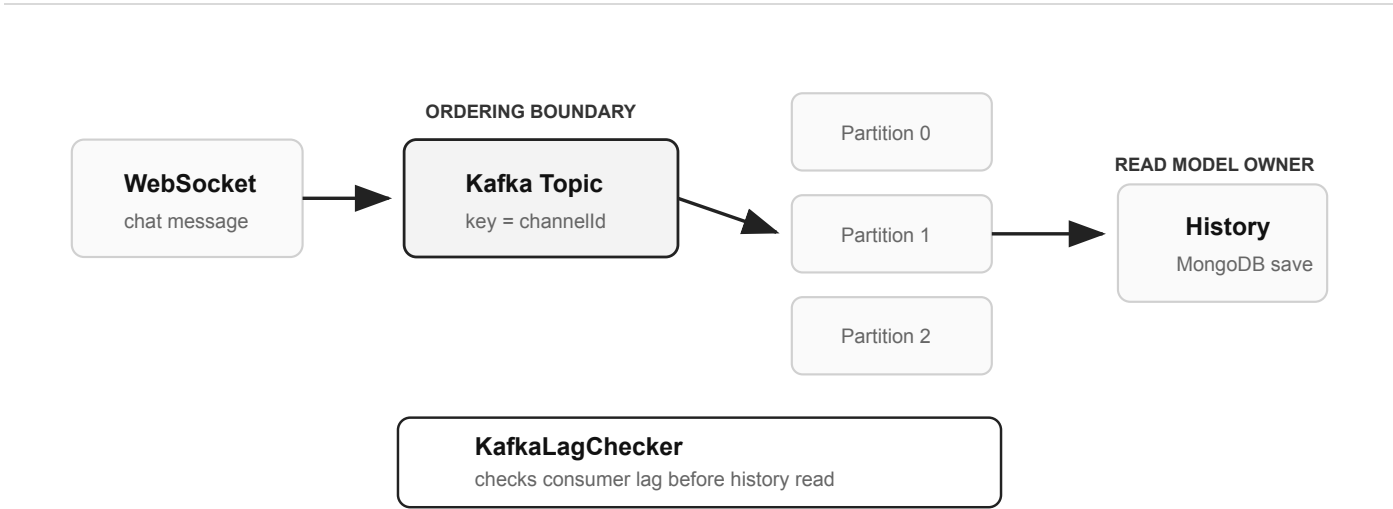
내 역할: Spring Cloud 초기 세팅, Kafka 설치/컨테이너화, WebSocket 채팅 서버, MongoDB History Server, KafkaLagChecker, FCM 알림 서버, 서버 간 JWT 연동, Docker Compose 환경 분리.

핵심 문제

채팅 시스템은 메시지를 보내는 것만으로 끝나지 않습니다. 다중 서버 전파, 채널 내 순서, 저장 완료 전 조회, 알림 전송까지 하나의 흐름으로 봐야 했습니다.

218ms → 34ms 조회 응답 개선 History API 팀 테스트 기준	200 → 300 동시 접속 확장 팀 부하 테스트 기준	5 × 100ms Lag Polling 최대 5회 offset lag 확인	Channel ID Ordering Boundary Kafka key 기준
--	--	---	---

Measurement note: 응답 시간과 동시 접속 수는 팀 부하 테스트 환경 기준입니다. 운영 트래픽 수치가 아니라 설계 개선 전후 비교 지표로 해석합니다.

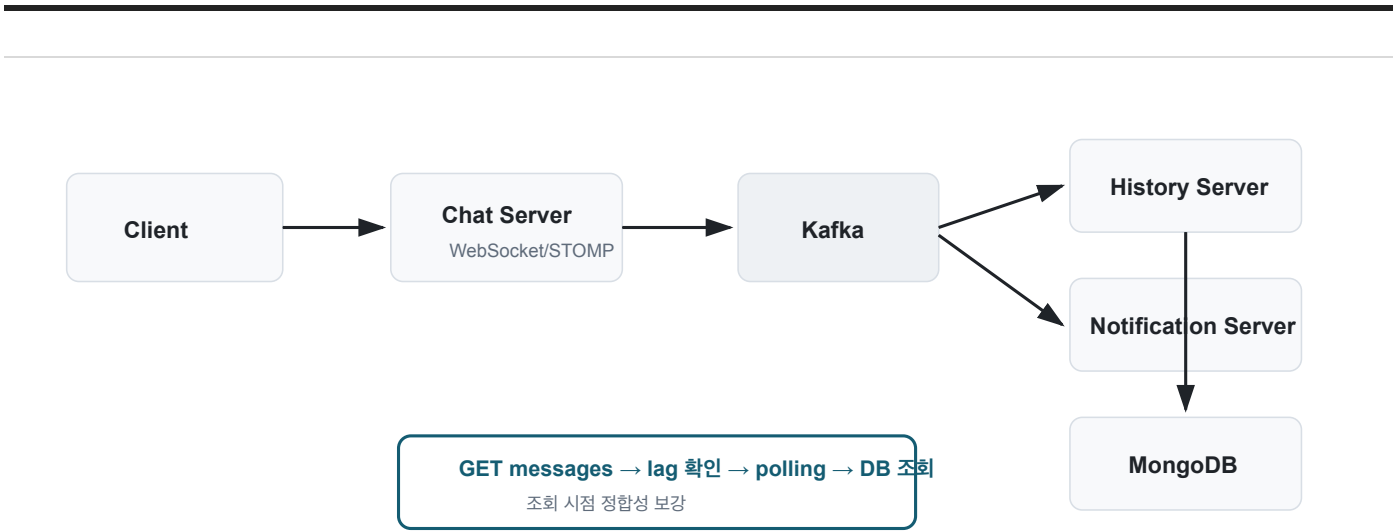


사용자가 체감하는 순서 보장 범위를 채널 단위로 정의하고, Channel ID를 Kafka key로 사용했습니다.

PROJECT 02 · SMILETOGETHER

02. SmileTogether

MSA 기반 협업 메신저 · Flow & Implementation



채팅 서버는 실시간 전달, History Server는 저장/조회, Notification Server는 알림 전송을 담당합니다.

KafkaLagChecker <pre>class KafkaLagChecker { lag = latestOffset - committedOffset if (lag > 0) retry(max=5, interval=100ms) }</pre> DB 조회 전에 Consumer 지연을 확인해, 비동기 저장으로 인한 조회 누락을 줄였습니다.	Channel Key Ordering <pre>producer.send(topic, { key: channelId, value: chatMessage })</pre> 전체 메시지가 아니라 같은 채널의 대화 순서만 보장하도록 ordering boundary를 좁혔습니다.
---	--

문제 1. 채팅 서버가 여러 대일 때 메시지를 어떻게 전파할 것인가

WebSocket 서버 간 직접 호출 대신 Kafka를 공유 이벤트 로그로 두어 서버 수 증가에 따른 전파 복잡도를 낮췄습니다.

PROBLEM

WebSocket 연결은 특정 인스턴스에 붙기 때문에 다중 서버에서 메시지 전파 경로가 끊길 수 있었습니다.

DECISION

서버 간 직접 호출 대신 Kafka topic을 서버 간 동기화 계층으로 선택했습니다.

CORE IMPLEMENTATION	STOMP handler	Kafka producer/consumer
---------------------	---------------	-------------------------

결론: 채팅 서버 확장 이후에도 메시지 전파 책임이 동일한 구조로 유지됐습니다.

PROJECT 02 · SMILETOGETHER

02. SmileTogether
MSA 기반 협업 메신저 · Problems & Decisions

문제 2. 채널 메시지 순서를 Kafka에서 어떻게 보장할 것인가

사용자가 체감하는 순서 보장을 “채널 내부”로 제한하고, Kafka key를 Channel ID로 결정했습니다.

PROBLEM

순서를 보장해야 하는 단위를 먼저 정하지 않으면 Kafka partition 전략이 흔들렸습니다.

DECISION

“같은 채널 안의 대화”를 ordering boundary로 정의했습니다.

CORE IMPLEMENTATION	channelId key	partition ordering
---------------------	---------------	--------------------

결론: Kafka ordering을 기술 기능이 아니라 사용자 대화 맥락 기준으로 설계했습니다.

문제 3. 비동기 저장 때문에 조회 시점에 메시지가 누락되는 문제

저장 완료를 DB 조회로 추측하지 않고 Kafka offset lag으로 확인한 뒤 히스토리 조회를 수행했습니다.

BEFORE

- 메시지 전송 직후 History API 조회 시 누락 가능
- DB만 읽으면 Consumer 지연을 알 수 없음

AFTER

- latest offset과 committed offset 차이 확인
- lag이 남으면 최대 5회, 100ms 간격 polling

PROBLEM

실제 유실이 아니어도 저장 지연은 사용자에게 메시지 유실처럼 보였습니다.

DECISION

조회 API가 Kafka consumer 지연을 인지한 뒤 MongoDB를 읽도록 했습니다.

CORE IMPLEMENTATION

KafkaLagChecker

latest offset

committed offset

결론: 비동기 저장 구조를 유지하면서 조회 시점의 정합성을 보강했습니다.

PROJECT 02 · SMILETOGETHER

02. SmileTogether

MSA 기반 협업 메신저 · Responsibility Boundary

문제 4. 실시간 전달과 히스토리 저장 책임을 어떻게 분리할 것인가

채팅 서버는 실시간 전달에 집중하고, 저장/조회/삭제 정책은 History Server로 분리했습니다.

PROBLEM

실시간 전송과 히스토리 관리는 데이터 모델과 변경 이유가 달랐습니다.

DECISION

Kafka 이벤트를 경계로 서버 책임을 분리했습니다.

CORE IMPLEMENTATION

History Server

MongoDB Document

결론: 채팅 서버 변경 없이 히스토리 조회 정책을 발전시킬 수 있는 구조가 됐습니다.

보조 구현

Profile/JWT: 채팅·히스토리 서버에서 고정 프로필 값을 제거하고, 서버 간 HTTP 호출과 HTTP/STOMP header 양쪽의

JWT 추출 흐름을 정리했습니다.

Notification: 채팅 서버가 알림 책임까지 가지지 않도록 Kafka 이벤트를 소비하는 Notification Server MVP와 FCM Web Push 전송 흐름을 분리했습니다.

Docker Compose: chat-server 단독, history-server 단독, 통합 실행 환경을 분리해 MSA 로컬 실행 재현성을 높였습니다.

KNOWN LIMITATION

offset lag polling은 조회 정합성을 높였지만 사용자를 잠깐 기다리게 만드는 방식입니다. Consumer 장애나 timeout 이후에는 idempotency, replay, fallback 응답 정책이 함께 있어야 운영 수준의 신뢰성을 확보할 수 있습니다.

실제 구현 Kafka 기반 WebSocket fan-out, Channel ID key ordering, KafkaLagChecker polling, History Server 분리	이후 고려한 방향 hot partition 감시, timeout fallback, consumer failure replay, idempotent message 처리
---	--

회고

Kafka를 사용한 이유는 메시지 큐 경험을 만들기 위해서가 아니라, 다중 서버 전파와 채널 단위 순서 보장이 필요했기 때문입니다. 특히 “전송 성공”과 “조회 정합성”은 다른 문제라는 점을 명확히 배웠습니다.

PROJECT 03 · FACEFRIEND

03. FaceFriend

이미지 분석 커뮤니티 채팅

2024.03 - 2024.06 · Backend · Spring Boot · WebSocket · STOMP · Redis · JPA

이 프로젝트를 한 줄로 설명하면

Redis를 실시간 상태 저장소로 사용해 채팅의 접속·읽음·실패 흐름을 명확히 한 프로젝트

1. 프로젝트 개요

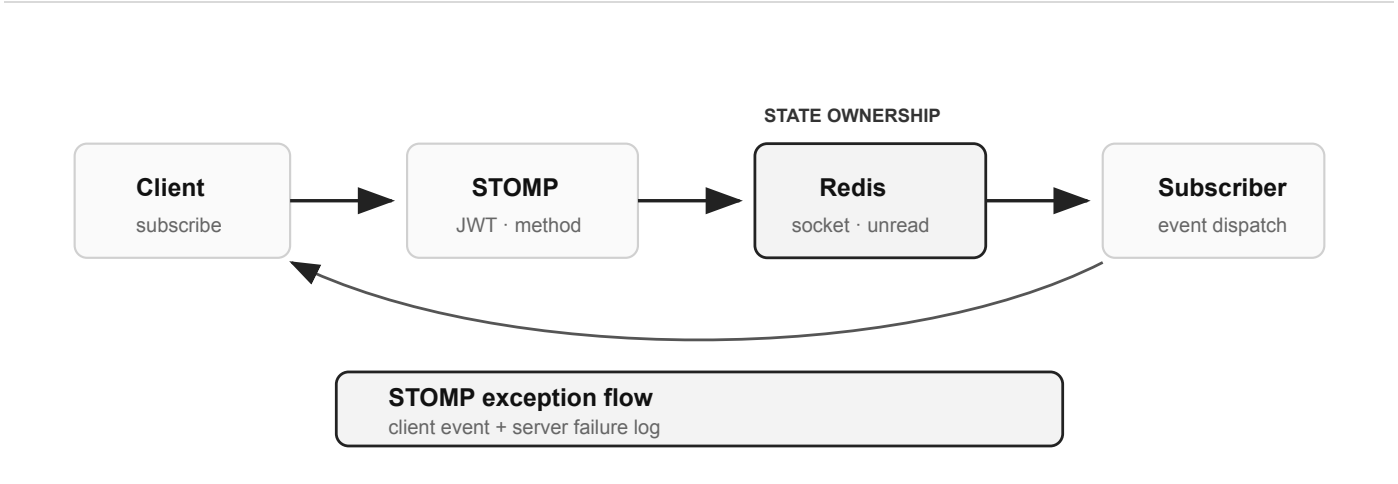
Spring Boot 기반 이미지 분석 커뮤니티에서 WebSocket/STOMP 채팅 기능과 채팅 응답 구조 개선을 담당했습니다.

내 역할: ChatRoom/ChatMessage 도메인, Redis Pub/Sub, Redis Repository 기반 socket/chat 상태 저장, unread 처리, STOMP 예외 응답, 채팅방 상태별 DTO.

핵심 문제

실시간 기능은 메시지를 보내는 코드보다 접속 상태, 예외, 미확인 메시지, 이벤트 구분을 잃지 않는 흐름이 중요했습니다.

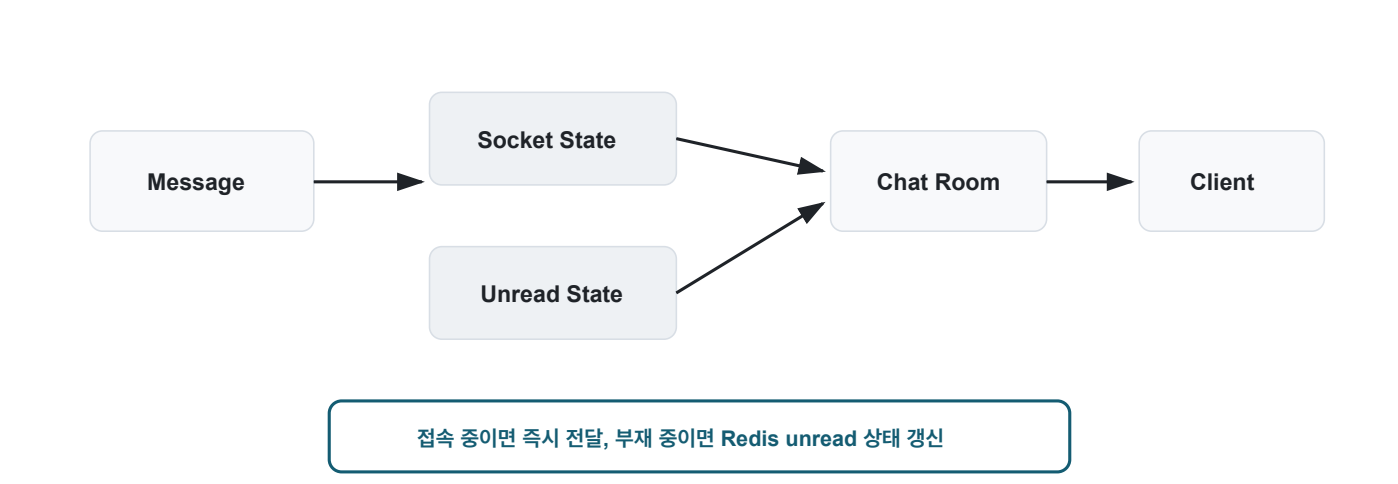
Redis State socket 접속 상태와 채팅방 입장 상태를 영속 DB와 분리	Unread Flow 상대의 접속/입장 상태에 따라 즉시 읽음 또는 미확인 증가 판단	STOMP Error 실패도 WebSocket 이벤트로 전달하면서 서버 예외 추적 유지
--	---	--



영속 메시지는 DB에, 접속/입장 상태는 Redis에 두어 실시간 판단과 저장 책임을 분리했습니다.

PROJECT 03 · FACEFRIEND

03. FaceFriend
이미지 분석 커뮤니티 채팅 · Redis State Flow



상대의 접속 상태와 채팅방 입장 상태에 따라 즉시 읽음 처리 또는 unread 증가를 결정합니다.

SocketInfoRedisRepository

```
class SocketInfoRedisRepository {
    save(userId, socketSession)
    isOnline(userId)
    remove(userId)
}
```

접속 상태를 영속 DB와 분리해 메시지 전송 시 실시간 판단에 사용했습니다.

STOMP Exception Flow

```
catch (exception) {
    send(errorDestination, errorPayload)
    keepServerExceptionLog(exception)
}
```

실패도 WebSocket 이벤트로 전달하면서 서버의 장애 추적 가능성을 유지했습니다.

PROJECT 03 · FACEFRIEND

03. FaceFriend
이미지 분석 커뮤니티 채팅 · Problems & Decisions

문제 1. 접속 상태와 채팅방 상태를 Redis에 어떻게 둘 것인가

접속 여부와 채팅방 입장 여부를 DB가 아니라 Redis의 실시간 상태로 분리했습니다.

BEFORE

- 메시지 저장과 접속 상태 판단이 섞임
- 상대가 방 안에 있는지 즉시 판단하기 어려움

AFTER

- 영속 메시지는 DB, 접속/입장 상태는 Redis
- 상태에 따라 즉시 읽음 또는 unread 증가

PROBLEM

접속 상태는 변경 빈도가 높고 오래 보관할 데이터가 아니었습니다.

DECISION

Redis를 캐시가 아니라 실시간 상태 저장소로 사용했습니다.

CORE IMPLEMENTATION

SocketInfoRedisRepository

ChatRoomInfoRedisRepository

unread flow

결론: 메시지 저장 책임과 실시간 상태 판단 책임을 분리했습니다.

문제 2. STOMP 예외가 프론트엔드에 전달되지 않는 문제

WebSocket 흐름에서는 실패도 이벤트로 전달해야 하므로, 클라이언트 알림과 서버 예외 추적을 함께 유지했습니다.

PROBLEM

실시간 메시징에서는 성공뿐 아니라 실패도 같은 흐름 안에서

DECISION

사용자에게는 STOMP 오류 이벤트를 보내고 서버에는 실패

다뤄야 했습니다.

흐름을 남겼습니다.

CORE IMPLEMENTATION	STOMP error response	exception tracking
---------------------	----------------------	--------------------

결론: 프론트엔드 사용자 경험과 서버 디버깅 가능성을 동시에 유지했습니다.

PROJECT 03 · FACEFRIEND

03. FaceFriend
이미지 분석 커뮤니티 채팅 · Event Contract

문제 3. 동일 채널의 여러 이벤트를 프론트가 구분하기 어려운 문제

같은 STOMP 채널로 내려가는 이벤트를 `method` 필드로 표준화해 프론트 분기 기준을 명확히 했습니다.

PROBLEM

실시간 이벤트는 endpoint만으로 의미가 구분되지 않았습니다.

DECISION

payload 내부에 이벤트 타입을 명시하는 방식으로 표준화했습니다.

CORE IMPLEMENTATION	method field	state DTO
---------------------	--------------	-----------

결론: 프론트엔드는 payload의 `method`만 보고 채팅 UI 상태를 갱신할 수 있게 됐습니다.

KNOWN LIMITATION

Redis 기반 실시간 상태는 빠르지만 네트워크 단절, 브라우저 종료, 재접속 시나리오에서 만료 정책과 복구 흐름이 중요합니다. 이후에는 connection TTL, heartbeat, 재접속 시 unread 재계산 정책을 더 명확히 둘 필요가 있습니다.

실제 구현 Redis socket/chat 상태 저장소, unread 판단 흐름, STOMP 예외 응답, method 기반 이벤트 표준화	이후 고려한 방향 heartbeat, connection TTL, reconnect recovery, unread 재계산 정책
---	---

회고

WebSocket은 연결만 열면 되는 기능이 아니었습니다. 접속 여부, 채팅방 입장 여부, 실패 이벤트, unread 상태를 각각 어떤 저장소와 DTO가 소유할지 정해야 유지보수가 가능했습니다.

04. 기타 프로젝트

상태, 흐름, 객체 책임, 협업 방식의 보조 근거

OneHabit · KiKi Kiosk · Wakeup · Relanz · Java GC Puzzle · Collaboration

협업

SCHEDULER

STATE

OneHabit · 시간에 따라 바뀌는 습관 상태

해커톤 일정에서 기능 범위를 재정의하고, 하루 1회 인증·연속 인증·마감 상태를 Scheduler 책임으로 분리했습니다. 짧은 기간에 팀이 같은 문제를 보도록 상태 기준을 문서화한 경험입니다.

DOMAIN

OOP

RESPONSIBILITY

KiKi Kiosk · Use Case에서 객체 책임 도출

기능 구현 전에 Use Case와 Sequence Diagram으로 객체 간 메시지를 먼저 정리하고, GRASP 관점에서 책임을 배치한 경험입니다.

FLOW

TRANSACTION

SERVICE

Wakeup Backend · 하나의 이벤트가 여러 상태로 전파되는 흐름

알람 확인 이벤트가 EXP, 포인트, 레벨, 통계 상태로 전파될 때 Service 계층에서 변경 순서와 트랜잭션 경계를 잡은 경험입니다.

EXPLICIT FLOW

DJANGO

SESSION

Relanz · Session 의존을 줄인 데이터 흐름

Session에 암묵적으로 의존하던 설문 리포트 데이터를 View에서 명시적으로 조회하도록 바꿔 화면 데이터 흐름을 안정화한 경험입니다.

CS

RUNTIME

MODELING

Java GC Puzzle · 런타임 상태를 시각 모델로 검증

Heap, Stack, 객체 참조, GC 대상 판별을 퍼즐 규칙으로 바꿔 런타임 상태 변화를 실행 감각으로 이해한 학습 프로젝트입니다.

TEAM

REVIEW

QUALITY GATE

Collaboration · 문서화와 품질 게이트

이슈 기반 설계 문서화, 오후 싱크, 최소 리뷰 규칙, AI 기반 테스트 코드와 CI 검증으로 팀의 의사결정 맥락과 회귀 검증 기준을 공유했습니다.